# Занятие 48.2. std::vector (векторы)

Ранее рассматривали std::array, который является более безопасной и удобной формой обычных фиксированных массивов в языке C++. Аналогично, в Стандартной библиотеке C++ есть и улучшенная версия динамических массивов (более безопасная и удобная) — std::vector.

В отличие от std::array, который недалеко отходит от базового функционала обычных фиксированных массивов, std::vector идет в комплекте с дополнительными возможностями, которые делают его одним из самых полезных и универсальных инструментов в языке C++.

**Оглавление:**


1. Векторы
2. Нет утечкам памяти!
3. Длина векторов
4. Заключение


## Векторы

Представленный в C++03, **std::vector** (или просто ***«вектор»***) — это тот же динамический массив, но который может сам управлять выделенной себе памятью. Это означает, что вы можете создавать массивы, длина которых задается во время выполнения, без использования операторов new и delete (явного указания выделения и освобождения памяти). std::vector находится в заголовочном файле vector. Объявление std::vector следующее:

```
#include <vector>

// Нет необходимости указывать длину при инициализации
std::vector<int> array;
std::vector<int> array2 = { 10, 8, 6, 4, 2, 1 }; // используется список инициализаторов для инициализации массива
std::vector<int> array3 { 10, 8, 6, 4, 2, 1 }; // используется uniform-инициализация для инициализации массива (начиная с C++11
```

Обратите внимание, что в неинициализированном, что в инициализированном случаях вам не нужно явно указывать длину массивов. Это связано с тем, что std::vector динамически выделяет память для своего содержимого по запросу.

Подобно std::array, доступ к элементам массива может выполняться как через оператор `[]` (который не выполняет проверку диапазона), так и через функцию at() (которая выполняет проверку диапазона):

```
array[7] = 3; // без проверки диапазона
array.at(8) = 4; // с проверкой диапазона
```

В любом случае, если вы будете запрашивать элемент, который находится вне диапазона `array`, длина вектора автоматически изменяться не будет. Начиная с C++11, вы также можете присваивать значения для std::vector, используя список инициализаторов:

```
array = { 0, 2, 4, 5, 7 }; // ок, длина array теперь 5
array = { 11, 9, 5 }; // ок, длина array теперь 3
```

В таком случае вектор будет самостоятельно изменять свою длину, чтобы соответствовать количеству предоставленных элементов.

## Нет утечкам памяти!

Когда переменная-вектор выходит из области видимости, то она автоматически освобождает память, которую контролировала (занимала). Это не только удобно (так как вам не нужно это делать вручную), но также помогает предотвратить утечки памяти. Рассмотрим следующий фрагмент:

```
void doSomething(bool value)
{
    int *array = new int[7] { 12, 10, 8, 6, 4, 2, 1 };

    if (value)
        return;

   // Делаем что-нибудь

    delete[] array; // если value == true, то этот стейтмент никогда не выполнится
}
```

Если переменной `value` присвоить значение `true`, то `array` никогда не будет удален, память никогда не будет освобождена и произойдет утечка памяти.

Однако, если бы `array` был вектором, то подобное никогда бы и не произошло, так как память освобождалась бы автоматически при выходе `array` из области видимости (независимо от того, выйдет ли функция раньше из области видимости или нет). Именно из-за этого использование std::vector является более безопасным, чем динамическое выделение памяти через оператор new.

## Длина векторов

В отличие от стандартных динамических массивов, которые не знают свою длину, std::vector свою длину запоминает. Чтобы её узнать, нужно использовать **функцию size()**:

```
#include <vector>
#include <iostream>

int main()
{
    std::vector<int> array { 12, 10, 8, 6, 4, 2, 1 };
    std::cout << "The length is: " << array.size() << '\n';

    return 0;
}
```

Результат:

```
The length is: 7
```

Изменить длину стандартного динамически выделенного массива довольно проблематично и сложно. Изменить длину std::vector так же просто, как вызвать **функцию resize()**:

```
#include <vector>
#include <iostream>

int main()
{
    std::vector<int> array { 0, 1, 2 };
    array.resize(7); // изменяем длину array на 7

    std::cout << "The length is: " << array.size() << '\n';

    for (auto const &element: array)
        std::cout << element << ' ';

    return 0;
}
```

Результат:

```
The length is: 7
0 1 2 0 0 0 0
```

Здесь есть две вещи, на которые следует обратить внимание. Во-первых, когда мы изменили длину `array`, существующие значения элементов сохранились! Во-вторых, новые элементы были инициализированы значением по умолчанию в соответствие с определенным типом данных (значением `0` для типа int).

Длину вектора также можно изменить и в обратную сторону (обрезать):

```
#include <vector>
#include <iostream>

int main()
{
    std::vector<int> array { 0, 1, 4, 7, 9, 11 };
    array.resize(4); // изменяем длину array на 4

    std::cout << "The length is: " << array.size() << '\n';

    for (auto const &element: array)
        std::cout << element << ' ';

    return 0;
}
```

Результат:

```
The length is: 4
0 1 4 7
```

Изменение длины вектора является затратной операцией, поэтому вы должны стремиться минимизировать количество подобных выполняемых операций.

## Заключение

Это вводная статья, предназначенная для ознакомления с основами std::vector. На следующих уроках мы детально рассмотрим std::vector, в том числе и разницу между длиной и ёмкостью вектора, и то, как в std::vector выполняется выделение памяти.

Поскольку переменные типа std::vector могут сами управлять выделенной себе памятью (что помогает предотвратить утечку памяти), отслеживают свою длину и легко её изменяют, то рекомендуется использовать std::vector вместо стандартных динамических массивов.